

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Využitie umelých neurónových sietí pre fúziu dát zo snímačov

Seminárna práca

Študijný program:	Robotika a kybernetika
Študijný odbor:	kybernetika
Školiace pracovisko:	Ústav robotiky a kybernetiky
Predmet:	D-PS2-RK – Predmet špecializácie Robotika a Kybernetika II
Školiteľ:	prof. Ing. František Duchoň, PhD.
Konzultant:	Ing. Slavomír Kajan, PhD.

Obsah

Úvod	5
1 Čo sú umelé neurónové siete	6
1.1 Aktivačná funkcia	8
2 Perceptróny	10
2.1 Jednovrstvové perceptróny	10
2.2 Lineárna separovateľnosť – problém klasifikácie	11
2.3 Viacvrstvové perceptróny	12
3 Trénovanie UNS	15
3.1 Perceptrónové kritérium	15
3.2 Spätné šírenie chyby	16
3.3 Problémy trénovania UNS	19
3.3.1 Pretrénovanie	19
3.3.2 Miznúce a explodujúce gradienty	20
3.3.3 Ťažkosti s konvergenciou	21
3.3.4 Lokálne a falošné optimá	22
3.3.5 Výpočtová náročnosť	22
3.4 Súčasné prístupy k trénovaniu	22
4 Hlboké učenie	25
4.1 Konvolučné neurónové siete	25
4.2 Rekurentné neurónové siete	27
5 Fúzia lokalizácie pomocou UNS	29
5.1 CNN fúzia	30
5.2 RNN fúzia	31
Záver	34
Literatúra	35

Zoznam obrázkov a tabuliek

Obrázok 1	McCulloch-Pittsov model neurónu.	6
Obrázok 2	Vybrané aktivačné funkcie.	9
Obrázok 3	Geometrické riešenie klasifikácie.	11
Obrázok 4	Lineárna separovateľnosť vybraných logických funkcií.	12
Obrázok 5	Viacvrstvový perceptrón.	13
Obrázok 6	Porovnanie klasifikácie pomocou SLP a MLP.	14
Obrázok 7	Základná schéma algoritmov tréningu UNS.	15
Obrázok 8	Schéma architektúry konvolučnej neurónovej siete. [18]	26
Obrázok 9	Rekurentná neurónová sieť – znázornenie toku informácií v čase.	27
Obrázok 10	Porovnanie štruktúr LSTM a GRU rekurentných neurónových sietí. [10]	28
Obrázok 11	Fúzia senzorov pomocou DFFN [2]	30

Zoznam značiek a skratiek

CNN	Convolutional Neural Network
DFFN	Deep Feed-Forward Network
EKF	Extended Kalman Filter
ELM	Extreme Learning Machine
GNSS	Global Navigation Satellite System
GPU	Graphical Processing Unit
GRU	Gated Recurrent Unit
IMU	Inertial Measurement Unit
INS	Inertion Navigation System
LSTM	Long Shor-Term Memory
MLP	Multi-Layer Perceptron
ReLU	Rectified Linear Unit
RMSE	Root Mean Square Error
RNN	Recurrent Neural Network
SGD	Stochastic Gradient Descent
SLAM	Simultaneous Localization and Mapping
SLP	Single-Layer Perceptron
SSD	Single Shot MultiBox Detector
UI	Umelá inteligencia
UNS	Umelá neurónová sieť

Úvod

Cieľom technológií umelej inteligencie (UI) je pomocou technických prostriedkov napodobniť biologické princípy ako ľudského organizmu, iných živočíchov a rastlín, ale aj prírodných zákonitostí ako takých. Výskum v oblasti UI začal už v prvej polovici 20. storočia, dlhé desaťročia boli však viaceré koncepty UI len na úrovni matematickej teórie, keďže vtedy známe technické prostriedky nedisponovali takým výpočtovým výkonom, ktorý by postačoval na efektívnu implementáciu týchto návrhov. Preto najmä v posledných desaťročiach, keď sa hranice výpočtových možností posúvajú závratným tempom ďalej, sme svedkami masívneho rozmachu v oblasti výskumu a aplikácií metód umelej inteligencie. Uplatnenie nachádzajú najmä v riešení komplexných úloh, ktoré sú problémové na analytický opis a riešenie konvenčnými algebraickými metódami.

Problematika fúzie dátových tokov z viacerých snímačov rovnakého alebo rôzneho typu je tiež jednou z možných aplikácií UI. Cieľom je aj v tomto prípade zabezpečiť stabilitu výsledného dátového toku, znížiť šum meraní či zamedziť negatívnym vplyvom skokovitej chyby meraní niektorého zo senzorov. Oproti klasickým metódam ako sú Kalmanove či časticové filtre, potenciál nástrojov UI je v rozpoznávaní zložitejších vzorov v senzorických dátach, schopnosti spracúvať veľké množstvo zdrojov dát, ale tiež v možnosti uskutočniť komplexnejší rozhodovací proces. Na to sa využívajú viaceré prístupy UI, ako sú umelé neurónové siete a ich modifikácie (hlboké, konvolučné, rekurzívne), strojové učenie a jeho modifikácie (učenie s posilňovaním, hlboké učenie) či fuzzy logika. V nasledujúcich častiach si predstavíme základný princíp fungovania týchto algoritmov a ukážeme aj príklady ich použitia v konkrétnych aplikáciách na fúziu údajov pre lokalizáciu.

1 Čo sú umelé neurónové siete

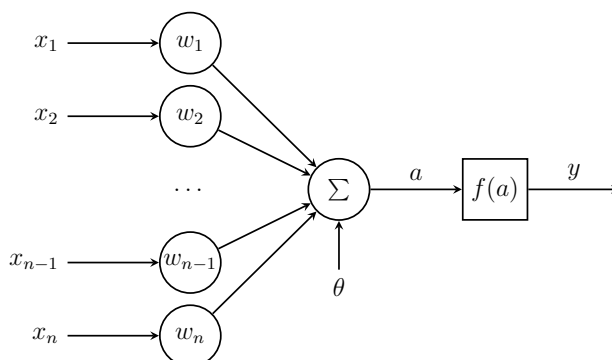
Cieľom umelých neurónových sietí (UNS) je napodobniť štruktúru a spôsob fungovania ľudského mozgu. Ten je na vonkajšej strane pokrytý mozgovou kôrou, ktorá obsahuje obrovské množstvo neurónov (v literatúre sa uvádza číslo približne 10^{10} [18]), ktoré sú vzájomne prepojené axónmi cez spojenia tvorené synapsami na jednej strane a dendridmi na druhej, vďaka čomu dochádza k prenosu informácií v podobe elektrického signálu. Jeden ľudský neurón môže byť prepojený s tisíckami iných [18]. Centrálna nervová sústava je schopná paralelne spracúvať veľké množstvo informácií z receptorov, pracovať s pamäťou a učiť sa z poznatkov, názorných aj analogických príkladov, zovšeobecňovať, konkretizovať a v konečnom dôsledku generovať riadiace pokyny pre ostatné časti tela.

Elementárnym prvkom UNS je *umelý neurón*. Zjednodušený model umelého neurónu, používaný v podstate dodnes, bol predstavený ešte v roku 1943 v práci McCullocha a Pittsa [17]. Jeho grafické znázornenie môžeme vidieť na obrázku 1. Matematicky je možné vyjadriť vstupno-výstupnú závislosť umelého neurónu ako

$$y = f\left(\sum_{i=1}^n w_i x_i - \theta\right) = f(a), \quad (1.1)$$

kde x_i sú vstupy do neurónu, w_i váhy synaptických prepojení smerujúcich do neurónu, θ je prah neurónu (nazývaný tiež *bias*), a predstavuje vnútornú aktivitu, $f(a)$ aktivačnú funkciu a y je výstup z neurónu.

UNS sa skladajú z množstva neurónov, ktoré sú usporiadané do vrstiev. Prvá vrstva sa nazýva vstupná, neuróny v nej prijímajú vstupné informácie a posielajú ich ďalej. Posledná vrstva neurónovej siete, ktorá poskytuje výsledok, sa nazýva výstupná. Medzi vstupnou a výstupnou vrstvou sa môžu vyskytovať tzv. skryté vrstvy, ktoré slúžia na vykonávanie medzivýpočtov, čím umožňujú riešenie komplexnejších úloh. Výpočtový



Obr. 1: McCulloch-Pittsov model neurónu. x_i sú vstupy neurónu, w_i váhy synaptických spojení, θ je prah neurónu, $f(a)$ predstavuje aktivačnú funkciu, a vnútornú aktivitu, y je výstup z neurónu.

výkon siete je daný práve schopnosťou skrytých vrstiev modelovať nelineárne vzťahy medzi vstupmi a výstupmi. UNS je možné rozdeliť do viacerých kategórií:

1. Podľa architektúry usporiadania neurónov do vrstiev:

- (a) *Jednovrstvové siete – perceptróny (SLP)* – skladajú sa len zo vstupnej (nevykonávajúcej výpočty) a výstupnej vrstvy; využíva sa úplné prepojenie, teda každý vstupný neurón je prepojený s každým výstupným; obvykle využívajú skokové aktivačné funkcie a ich účelom je klasifikácia do tried;
- (b) *Viacvrstvové siete – viacvrstvové perceptróny (MLP)* – obsahujú aspoň jednu skrytú vrstvu; vstupná vrstva slúži iba na distribúciu vstupných informácií, výkonnú funkciu majú iba skryté vrstvy a výstupná vrstva. Umožňujú riešenie lineárne neseparovateľných problémov.
- (c) *Hlboké neurónové siete* – majú vyšší počet skrytých vrstiev, umožňujú modelovanie komplexnejších závislostí. Medzi najznámejšie architektúry patria hlboké dopredné neurónové siete (DFFN), konvolučné neurónové siete (CNN), používané v oblasti spracovania obrazu, a rekurentné neurónové siete (RNN), vhodné na spracovanie sekvenčných dát [15, 23].

2. Podľa spôsobu šírenia informácií v rámci siete:

- (a) *Dopredné siete* – (angl. *feed-forward*) obsahujú iba väzby v smere od vstupov k výstupom,
- (b) *Rekurentné siete* – obsahujú aspoň jednu spätnú väzbu v rámci tej istej vrstvy alebo v rámci rôznych vrstiev,

Kľúčom k úspešnosti neurónových sietí vo vykonávaní učenej úlohy je proces ich učenia. Učenie je možné definovať ako proces zlepšovania výkonu na základe pozorovania okolitého prostredia [22]. Proces učenia neurónovej siete zahŕňa úpravu váh w_i na základe chyby medzi predikovaným a skutočným výstupom. Najčastejšie sa používa algoritmus *gradientného zostupu* v kombinácii s technikou *spätného šírenia* (angl. *backpropagation*) [21]. Cieľom je minimalizovať hodnotu funkcie odchýlky, napr. strednej kvadratickej chyby:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (1.2)$$

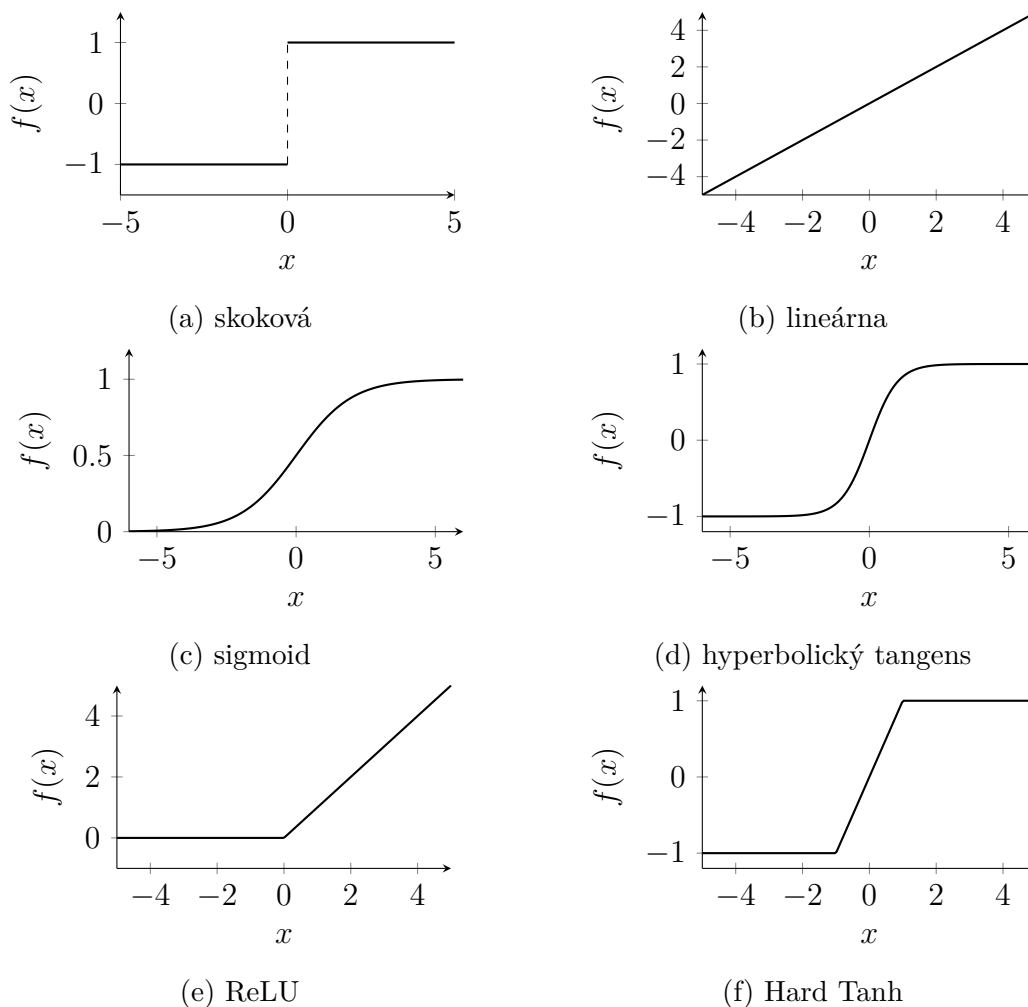
kde y_i je skutočný a $\hat{y}_i$ predikovaný výstup.

1.1 Aktivačná funkcia

Aktivačná funkcia zabezpečuje nelinearitu neurónu, vďaka čomu je UNS schopná riešiť zložitejšie problémy. Výber vhodnej aktivačnej funkcie je jednou z kritických častí návrhu UNS a musí byť prispôsobený typu úlohy, ktorú má neurónová sieť vykonávať. V pôvodnom modeli [17] sa používala len binárna (skoková) funkcia $f(a) = \text{sgn}(a)$. Tá je stále používaná v perceptrónových neurónových sieťach, ktoré vykonávajú klasifikáciu. V mnohých prípadoch sa však vyžaduje spojitý výstup siete, preto sa používajú viaceré typy spojitých aktivačných funkcií [1]:

- *lineárna funkcia*: $f(a) = a$ – používa sa v prípade, že výstupná premenná je reálne číslo; v takom prípade rieši sieť regresiu najmenších štvorcov $\hat{y} = \sum_{i=1}^n w_i x_i$;
- *sigmoida*: $f(a) = S(a) = \frac{1}{1+e^{-\beta a}}$ – vhodná pre prípad odhadu pravdepodobnosti binárnej triedy, účelová funkcia má tvar $\hat{y} = -\log\left|\frac{y}{2} - 0,5 + \hat{y}\right|$ za predpokladu $y \in \{-1, 1\}$;
- *hyperbolický tangens*: $f(a) = \tanh\left(\frac{a}{2}\right) = \frac{1-e^{-a}}{1+e^{-a}} = 2S(2a) - 1$ – ide v podstate o preškálovaný a transformovaný sigmoid tak, že pripomína skokovú funkciu, avšak na rozdiel od nej je diferencovateľná. Je vhodná v prípade, že výstup môže nadobúdať kladné aj záporné hodnoty. Navyše, jej gradient je viac koncentrovaný okolo nuly, vďaka čomu sa trénuje jednoduchšie než sigmoid. Tento efekt sa zvykne ešte zosilniť modifikáciami, ako napr. $f(a) = \tanh(3a)$;
- *upravená lineárna jednotka (ReLU)*: $f(a) = \max\{a, 0\}$ – patrí k novším typom aktivačných funkcií; je výhodná pri trénovaní viacvrstvových sietí s množstvom neurónov, a to najmä vďaka rýchlejšiemu výpočtu gradientu, ale tiež pre nižšie riziko výskytu *efektu miznúceho gradientu* a zabránenie saturácii. Nevýhodou je strata informácie zo záporných vstupov a riziko vzniku tzv. *mŕtveho neurónu*: keď sa raz váhy neurónu natrénujú na záporné hodnoty, jeho výstup bude vždy nulový;
- „*Hard Tanh*“: $f(a) = \max\{\min[v, 1], -1\}$ – je to po častiach lineárna aproximácia hyperbolického tangensu; výpočtovo menej náročná, zachováva saturáciu, čo býva žiaduce napríklad pri výstupných neurónoch; podobne ako ReLU má v citlivej časti konštantný nenulový gradient, čo má význam pri spätnom šírení gradientu.

Aktivačné funkcie ReLU a Hard Tanh v poslednej dobe zväčša nahrádzajú klasické typy, ako sú sigmoid a hyperbolický tangens, a to pre zjednodušenie trénovania viacvrstvových neurónových sietí s týmito funkciami [1]. Grafické znázornenia aktivačných funkcií sa nachádzajú na obr. 2.



Obr. 2: Vybrané aktivačné funkcie.

V MLP sieťach sa v neurónoch výstupnej vrstvy často využíva aj aktivačná funkcia *softmax*. Jej úlohou je prepočítať výstupy poslednej skrytej vrstvy na pravdepodobnostné hodnoty, teda musí platiť $y_i \in \langle 0; 1 \rangle$ a zároveň $\sum_i y_i = 1$. To je zabezpečené predpisom funkcie

$$f(y_i) = \frac{\exp(h_i)}{\sum_{j=1}^k \exp(h_j)} ; \forall i \in \{1, \dots, k\}, \quad (1.3)$$

kde $H = (h_1, \dots, h_k)^\top$ je vektor výstupov poslednej skrytej vrstvy a k je počet výstupných neurónov aj neurónov poslednej skrytej vrstvy. Tento typ aktivačnej funkcie je výhodné použiť na klasifikáciu väčšieho množstva tried [1].

2 Perceptróny

Základným typom neurónových sietí sú perceptróny. Slúžia na klasifikáciu vstupných údajov a pôvodne boli predstavené predstavené Rosenblattom v roku 1958 [20], ktorý ich architektúru rozdelil na tri vrstvy: senzorickú, asociačnú a výstupnú. Senzorická vrstva sa namiesto plnohodnotných neurónov skladá len zo vstupných terminálov, ktorých počet zodpovedá rozmeru vstupného vektora a ich úlohou je preposielanie vstupných hodnôt ďalej (príp. normalizácia vstupných hodnôt). Za ňou môže nasledovať asociačná, teda skrytá vrstva, v ktorej neuróny slúžia na vykonávanie medzivýpočtov a umožňujú riešenie lineárne neseparovateľných problémov (viac v časti 2.2). Výstupy skrytej vrstvy nie sú čitateľné, slúžia ako vstupy pre neuróny poslednej, výstupnej vrstvy. V pôvodnej architektúre váhy skrytej vrstvy nepodliehali učeniu. Napokon, výstupná vrstva slúži na výsledné výpočty siete. Počet neurónov v nej zodpovedá počtu tried, ktoré sa v rámci úlohy klasifikácie rozlišujú. Neuróny v takejto sieti sú pospájané v doprednom smere (angl. *feed-forward*), teda zľava doprava. Všetky prepojenia perceptrónu sú jednosmerné, informácie sa posielajú len v smere dopredu k neurónom susediacim sprava. Nenachádzajú sa v ňom žiadne spätné prepojenia ani prepojenia preskakujúce vrstvu [18].

Počas nasledujúceho vývoju v oblasti UNS však došlo z didaktických dôvodov k ustáleniu termínu *perceptrón* iba na jednovrstvovú lineárne separovateľnú architektúru, *jednovrstvový perceptrón (SLP)*. Klasifikačné siete s menej ako štyrmi skrytými vrstvami sa zvyknú označovať aj *viacvrstvové perceptróny (MLP)*.

2.1 Jednovrstvové perceptróny

Vstupná vrstva jednovrstvého perceptrónu obsahuje d vstupných terminálov pre vstupný vektor $X = (x_1, x_2, \dots, x_d)^\top$, ku ktorým je priradený vektor váh $W = (w_1, w_2, \dots, w_d)^\top$. Výstupná jednotka vypočíta vážený súčet vstupov a výsledok spracuje pomocou skokovej funkcie, ktorá určuje výstupnú triedu [1]:

$$\hat{y} = \text{sign}(W \cdot X + \theta) = \text{sign} \left(\sum_{j=1}^d w_j x_j + \theta \right), \quad (2.1)$$

kde θ predstavuje tzv. *bias* – prahovú hodnotu, ktorá umožňuje posun rozhodovacej hranice. V implementáciách sa často reprezentuje ako špeciálny vstup s hodnotou 1 a vlastnou váhou.

Perceptrón funguje ako lineárny klasifikátor: rozhodovaciu hranicu tvorí hyper-rovina $W \cdot X + \theta = 0$, ktorá rozdeľuje vstupný priestor na dve triedy. Tento prístup je účinný v prípadoch, keď sú dáta *lineárne separovateľné*.

2.2 Lineárna separovateľnosť – problém klasifikácie

Problém lineárnej separovateľnosti je možné priblížiť na jednoduchom príklade. Majme klasifikačnú úlohu pre dvojrozmerné reálne vstupné vektory $X = (x_1, x_2)^\top$, ktoré majú byť rozdelené do cieľových skupín, teda tried $y \in \{-1, 1\}$. Na riešenie teoreticky stačí použiť jeden neurón so skokovou aktivačnou funkciou, teda výstup neurónu je počítaný ako skalárny súčin vstupov a váh:

$$\hat{y} = \text{sgn}(w_1 x_1 + w_2 x_2 - \theta) = \text{sgn}(W \cdot X - \theta), \quad (2.2)$$

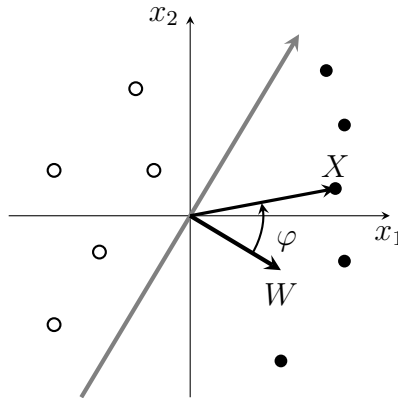
kde $W = (w_1, w_2)$ je vektor váh, ktoré pripájajú vstupy neurónu [18]. Skalárny súčin, ktorý vystupuje vo vzťahu 2.2, je možné riešiť aj geometricky po úprave:

$$\begin{aligned} W &= |W| \begin{bmatrix} \cos \alpha \\ \sin \alpha \end{bmatrix} ; |W| = \sqrt{w_1^2 + w_2^2}, \\ X &= |X| \begin{bmatrix} \cos \beta \\ \sin \beta \end{bmatrix} ; |X| = \sqrt{x_1^2 + x_2^2}, \\ W \cdot X &= |W||X| \cos(\alpha - \beta) = |W||X| \cos \varphi, \end{aligned} \quad (2.3)$$

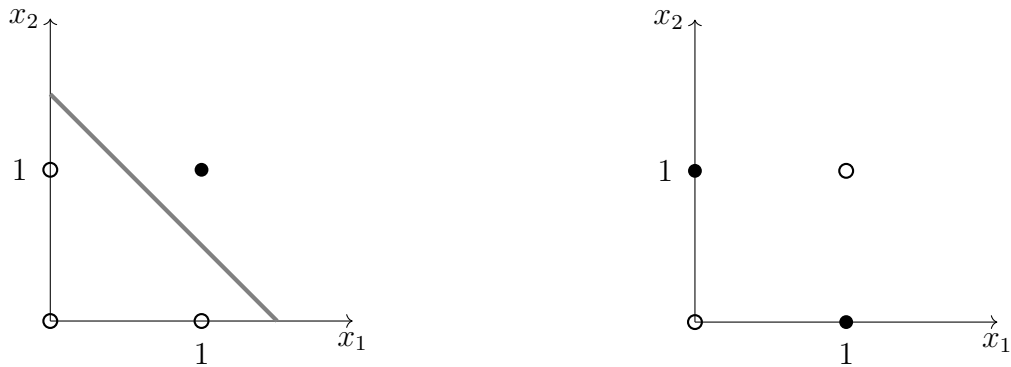
kde α a β sú uhly, ktoré zvierajú vektor váh, resp. vstupov s osou x_1 a φ je uhol medzi nimi. Z toho je zrejmé, že výsledok skalárneho súčinu bude kladný pre $\varphi \in (-\frac{\pi}{2}, \frac{\pi}{2})$, inak bude záporný. Dôsledkom toho je zabezpečená správna klasifikácia vtedy, keď je vektor váh kolmý na rozhodovaciu hranicu, ktorá vymedzuje jednotlivé triedy. Situácia je graficky znázornená na obr. 3.

V uvedenom príklade sme pre zjednodušenie neuvažovali s biasom neurónu θ . Jeho vplyv na rozhodovaciu hranicu je však zrejmý keď položíme argument zo vzťahu 2.2 rovný nule a odvodíme

$$x_2 = -\frac{w_1}{w_2}x_1 + \frac{\theta}{w_2}, \quad (2.4)$$



Obr. 3: Geometrické riešenie klasifikácie.



(a) Kojunkcia (AND) – lineárne separovateľná

(b) Exkluzívna disjunkcia (XOR) – lineárne neseparovateľná

Obr. 4: Lineárna separovateľnosť vybraných logických funkcií. • pre pravd. hodnotu „TRUE“ ($y = 1$), ○ pre pravd. hodnotu „FALSE“ ($y = -1$).

z čoho vyplýva, že bias má vplyv na priesečník rozhodovacej hranice s osou x_2 [18].

Ak existuje taká lineárna hranica (napr. priamka v 2D, rovina v 3D, hyper-rovina vo vyšších dimenziách), ktorá dokáže oddeliť všetky vstupné prvky jednej triedy od druhej bez chyby, takúto množinu vstupov nazývame *lineárne separovateľná* a jej klasifikácia je zvládnuteľná pomocou jediného jediného neurónu.

Dobрым príkladom na demonštráciu lineárnej separovateľnosti sú aj logické funkcie. Na obr. 4 je porovnanie funkcií konjunkcie, ktorá je lineárne separovateľná a exkluzívnej disjunkcie, ktorá nie je.

2.3 Viacvrstvové perceptróny

V prípade SLP je jediná výpočtová vrstva výstupná, keďže vstupná vrstva nevykonáva žiadne výpočty. Viacvrstvové perceptróny (MLP) obsahujú viacero výpočtových vrstiev. Keďže ale výsledky vrstiev pridaných medzi vstupnú a výstupnú vrstvu nie sú viditeľné pre užívateľa, označujú sa ako *skryté vrstvy*.

Pridanie skrytej vrstvy umožňuje MLP riešiť lineárne neseparovateľné problémy, nakoľko rozdeľuje klasifikačný priestor na viac než len dva segmenty. Jednoduchý príklad takejto siete, ktorá klasifikuje dvojrozmerný vstupný vektor pomocou troch skrytých a jedného výstupného neurónu, môžeme vidieť na obr. 5. Neuróny vypočítavajú výstupné hodnoty pomocou skokovej aktivačnej funkcie v tvare

$$h_j = f \left(\sum_k w_{jk} x_k - \theta_j \right), \quad (2.5a)$$

$$\hat{y}_i = f \left(\sum_j W_{ij} h_j - \theta_i \right), \quad (2.5b)$$

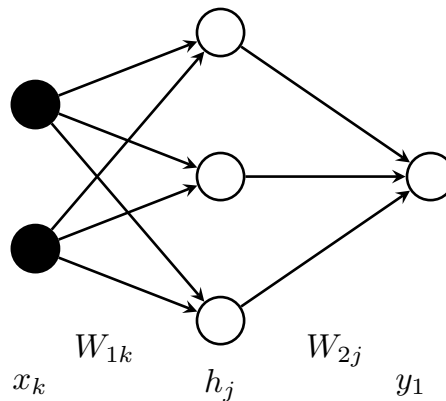
pričom výsledok môže nadobúdať hodnoty $y_i = \{-1, +1\}$ [18]. Každý z neurónov v skrytej vrstve má vlastnú rozhodovaciu hranicu. Cieľom je teda nájsť také hodnoty w_{jk} a θ_j , aby rozhodovacie hranice rozdelili vstupnú rovinu (keďže máme dvojrozmerný vstup, môžeme si ho predstaviť ako súradnice bodov v rovine) na také segmenty, ktoré budú obsahovať len prvky jednej triedy. Situácia je znázornená na obr. 6.

Mechanizmus klasifikácie je možné si predstaviť tak, že každý zo vstupných bodov obdrží množinu výstupných hodnôt skrytých neurónov $\{h_1, h_2, h_3\}$. Každý zo segmentov je potom charakterizovaný unikátnou kombináciou výstupov skrytej vrstvy. Váhy w_{1j} a prahové hodnoty θ_j musia byť nastavené tak, aby každému vzniknutému segmentu vstupnej roviny pridelil výstupný neurón správnu triedu y_i , čo už opäť predstavuje lineárne separovateľný problém. To prebieha v tomto konkrétnom prípade na základe vzťahu

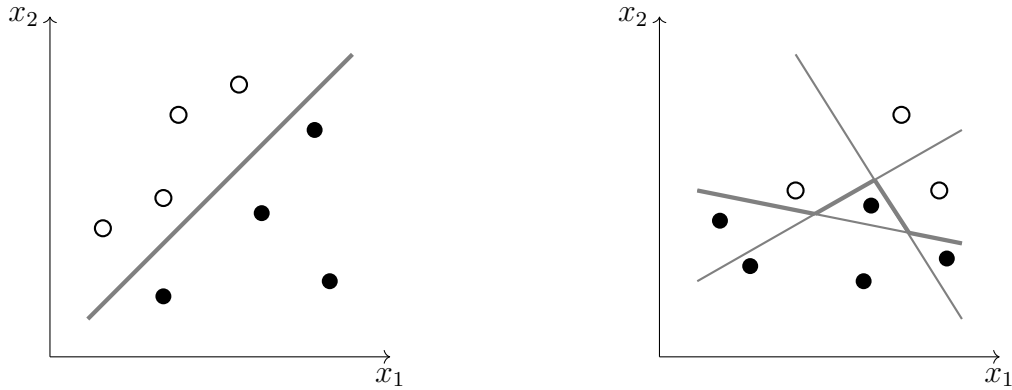
$$\hat{y}_1 = \text{sgn}(h_1 + h_2 + h_3). \quad (2.6)$$

Je potrebné si uvedomiť, že neexistuje jedinečné riešenie takéhoto problému; sú možné viaceré kombinácie váh a prahových hodnôt neurónov, ktoré by spĺňali požiadavky klasifikácie, ako aj viaceré rozloženia siete. Vo všeobecnosti však platí, že pre zložitejšie problémy klasifikácie je potrebných viac skrytých neurónov [18]. V práci autorov Minsky a Papert z roku 1969 [19] bolo preukázané, že všetky logické operácie je možné vyriešiť pomocou MLP za podmienky, že aspoň jeden zo skrytých neurónov je pripojený ku všetkým vstupným terminálom.

Pri základnej architektúre dopredných sietí sa predpokladá, že všetky neuróny jednej vrstvy sú pripojené ku všetkým neurónom nasledujúcej vrstvy. V prípade, že sieť obsahuje len jednu skrytú vrstvu, hovoríme o tzv. *plytkej sieti*. MLP však môže obsahovať viac než len jednu skrytú vrstvu. Je matematicky dokázané, že UNS s tromi vrstvami



Obr. 5: Viacvrstvový perceptrón s jednou skrytou vrstvou so vstupnými terminálmi x_k , skrytými neurónmi h_j , výstupným neurónom y_1 , a váhami prepojení medzi vrstvami w_{jk} a w_{1j} . [18]



(a) Lineárne separovateľný problém klasifikovaný pomocou SLP.

(b) Lineárne neseparovateľný problém klasifikovaný pomocou MLP, tvoriacej po častiach lineárnu rozhodovaciu hranicu.

Obr. 6: Porovnanie klasifikácie pomocou SLP a MLP.

sú schopné aproximovať ľubovoľnú spojitú nelineárnu funkciu a siete so štyrmi vrstvami stačia na aproximáciu ľubovoľnej nespojitej nelineárnej funkcie. Ak sieť obsahuje viac ako tri skryté vrstvy, zvykne sa nazývať *hlboká (dopredná) sieť (DFFN)*.

Pre praktický zápis matematických operácií vnútri siete sa na označenie jednotlivých prvkov používa vektorová, resp. maticová forma. MLP obsahuje vstupnú vrstvu označenú vektorom X veľkosti $d \times 1$, kde d je rozmer vstupných dát, k skrytých vrstiev o veľkosti $p_1 \dots p_k$, ktorých výstupy označujú vektory $H_1 \dots H_k$. Vstupná vrstva je na prvú skrytú vrstvu pripojená váhami označenými maticou W_1 veľkosti $d \times p_1$, zatiaľ čo r -tá skrytá vrstva je prepojená s $(r+1)$ -ou maticou W_r veľkosti $p_r \times p_{r+1}$. Za predpokladu, že máme vektor výstupných neurónov $O = (o_1, \dots, o_m)^\top$, tie sa pripájajú k poslednej skrytej vrstve pomocou váh označených maticou W_{k+1} s veľkosťou $p_k \times m$. Vstupno-výstupný výpočet MLP je potom možné zapísať pomocou nasledujúcich rekurzívnych vzťahov [1]:

$$H_1 = f(W_1^\top X) \quad (2.7a)$$

$$H_{p+1} = f(W_{p+1}^\top H_p) \quad \forall p \in (1 \dots k - 1) \quad (2.7b)$$

$$o = f(W_{k+1}^\top H_k). \quad (2.7c)$$

Základnými parametrami pri návrhu siete sú teda počet skrytých vrstiev, počty skrytých vrstiev a neurónov v jednotlivých vrstvách, a tiež výber aktivačných funkcií neurónov. V neposlednom rade, úspešnosť trénovania neurónovej siete závisí od výberu optimalizačnej funkcie odchýlky.

3 Trénovanie UNS

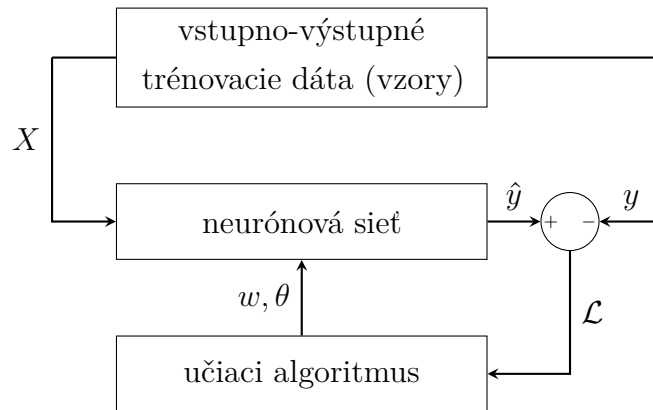
Trénovanie umelých neurónových sietí predstavuje kľúčový proces, prostredníctvom ktorého sa model učí z dostupných dát optimalizáciou svojich parametrov, najmä váh a prahov neurónov. Cieľom trénovania je minimalizovať odchýlku medzi výstupom siete $\hat{y}$ a požadovaným výstupom y , čo sa dosahuje iteratívnym prispôbovaním parametrov na základe výpočtu gradientu chyby. Jedným z kľúčových aspektov trénovania je preto výber vhodnej funkcie na výpočet odchýlky (v angl. literatúre označovanej *loss function*) vzhľadom ku konkrétnemu typu úlohy, ktorú UNS rieši. Obrázok 7 znázorňuje základný princíp trénovania UNS. V tejto časti sa podrobnejšie pozrieme na jeden z prvých používaných algoritmov – perceptrónové kritérium – ale aj na v súčasnosti najbežnejšiu metódu spätného šírenia chyby. Priblížime tiež časté problémy, ktoré sa pri procese trénovania môžu vyskytnúť.

3.1 Perceptrónové kritérium

Pôvodný perceptrónový algoritmus prebiehal heuristicky pomocou elektrických obvodov, na rozdiel od súčasného chápania problému ako optimalizačnej úlohy. V každom prípade, cieľ ostáva zachovaný a je ním nájsť také váhy W , ktoré minimalizujú počet chybne klasifikovaných príkladov v množine trénovacích vstupno-výstupných párov (X, y) . Úlohu perceptrónového algoritmu je teda možné zapísať v tvare účelovej funkcie najmenších štvorcov pre všetky trénovacie objekty datasetu $\mathcal{D}$ [1] ako

$$\mathcal{L} = \arg \min_W \sum_{(X,y) \in \mathcal{D}} (y - \hat{y})^2 = \sum_{(X,y) \in \mathcal{D}} [y - \text{sign}(W \cdot X)]^2 \quad (3.1)$$

Tento typ minimalizácie účelovej funkcie sa označuje aj ako *funkcia odchýlky* (z angl. *loss function*) [1]. Problém takto formulovanej funkcie však spočíva v použití skokovej



Obr. 7: Základná schéma algoritmov trénovania UNS.

funkcie signum, ktorá nie je diferencovateľná, obsahuje skoky a na veľkej časti definičného oboru má konštantné hodnoty, čo spôsobuje nulový gradient, teda nie je vhodná pre metódu gradientného zostupu. Na odvodenie jej hladkej náhrady sa používa tzv. *perceptrónové kritérium*. Jeho princíp spočíva v prvotnom definovaní účelovej funkcie s výstupom 0/1 pre jeden príklad vstupno-výstupného páru (X, y) [1]:

$$L^{(0/1)}(X, y) = \frac{1}{2} (y - \hat{y})^2 = 1 - y \cdot \text{sign}(W \cdot X + \theta). \quad (3.2)$$

Táto funkcia nie je diferencovateľná kvôli prítomnosti skokovej funkcie sign, preto sa na analytické účely nahrádza hladkou účelovou funkciou [1]:

$$L(X, y) = \max \{-y(W \cdot X + \theta), 0\}. \quad (3.3)$$

Táto tzv. *hladká náhrada funkcie odchýlky* penalizuje len nesprávne klasifikované príklady a jej gradient možno využiť na aktualizáciu váh [1]:

$$W \leftarrow W + \alpha (y - \hat{y}) X, \quad (3.4)$$

kde α je rýchlosť učenia. Váhy sa menia iba v prípade, že $y \neq \hat{y}$. Tento prístup je stochastickým gradientným zostupom (SGD), ktorý optimalizuje (aj keď len približne) uvedenú hladkú funkciu.

3.2 Spätné šírenie chyby

Optimalizačné kritérium pre SLP je ľahko definovateľné, keďže výstup siete je priamou funkciou váh. To umožňuje jednoduchý výpočet gradientu pomocou diferenciácie. Pri viacvrstvových sieťach je však odchýlka výstupu komplikovanou zloženou funkciou váh v predošlých vrstvách, ktorú môžeme zjednodušene zapísať ako $\mathcal{L} = f(o(h_r(w_r)))$. Preto sa za účelom výpočtu jej gradientu používa algoritmus *spätného šírenia chyby*. Ten využíva reťazové pravidlo diferenciálneho počtu, ktoré počíta gradienty chyby ako sumu násobkov lokálnych gradientov skrytých neurónov pozdĺž rôznych ciest od daného neurónu až po výstup. Keďže počet ciest medzi skrytým neurónom a výstupom môže rásť exponenciálne s hĺbkou a šírkou siete, priamy výpočet všetkých príspevkov by bol výpočtovo náročný. Preto sa spätné šírenie realizuje efektívne pomocou dynamického programovania, ktoré opakovane (rekurzívne) využíva už vypočítané medzivýsledky. Výpočet spätného šírenia chyby prebieha vo viacerých cykloch, tzv. *epochách*, pričom každá pozostáva z dvoch hlavných fáz:

1. *Dopredná*: Cez sieť sa v smere od vstupu k výstupu postupne prepočítavajú aktivácie všetkých neurónov pre každú tréningovú vzorku na základe aktuálneho

nastavenia váh. Výstup siete je porovnaný s požadovaným výstupom tréningových dát, na základe čoho sa vypočíta odchýlka výstupu.

2. V každej vrstve v smere od výstupu k vstupu sa vypočíta lokálny gradient odchýlky voči daným váham a aktiváciám z predchádzajúcej vrstvy, na základe čoho sú prepočítané jednotlivé hodnoty váh.

Uvažujme sekvenciu skrytých neurónov $h_1 \dots h_k$, ktorá je zakončená výstupným neurónom o . Keď budeme uvažovať len jedinú cestu z neurónu h_1 do o , potom je možné gradient váh smerujúcich do r -tého neurónu, ktoré označíme $w_{(h_{r-1}, h_r)}$, vypočítať pomocou reťazového pravidla jednej premennej [1]:

$$\frac{\partial \mathcal{L}}{\partial w_{(h_{r-1}, h_r)}} = \frac{\partial \mathcal{L}}{\partial o} \cdot \left[\frac{\partial o}{\partial h_k} \prod_{i=r}^{k-1} \frac{\partial h_{i+1}}{\partial h_i} \right] \cdot \frac{\partial h_r}{\partial w_{(h_{r-1}, h_r)}} \quad \forall r \in 1 \dots k \quad (3.5)$$

Vo všeobecnosti však bude v MLP sieti existovať exponenciálne väčšia množina prepojení neurónov h_r a o , ktorú označíme $\mathcal{P}$. Vzťah 3.5 je teda možné zovšeobecniť pre všetky existujúce prepojenia pomocou reťazového pravidla viacerých premenných:

$$\frac{\partial \mathcal{L}}{\partial w_{(h_{r-1}, h_r)}} = \frac{\partial \mathcal{L}}{\partial o} \cdot \underbrace{\left[\sum_{[h_r, h_{r+1}, \dots, h_k, o] \in \mathcal{P}} \frac{\partial o}{\partial h_k} \prod_{i=r}^{k-1} \frac{\partial h_{i+1}}{\partial h_i} \right]}_{\Delta(h_r, o) = \frac{\partial \mathcal{L}}{\partial h_r}} \cdot \frac{\partial h_r}{\partial w_{(h_{r-1}, h_r)}} \quad (3.6)$$

Tento vzťah je logicky rozdelený na dve časti. Prvá časť, označená ako $\Delta(h_r, o) = \frac{\partial \mathcal{L}}{\partial h_r}$, predstavuje agregovaný gradient chyby vzhľadom na výstup neurónu h_r a zahŕňa vplyv všetkých ciest vedúcich z tohto neurónu až po výstup o . Nakoľko jeho veľkosť exponenciálne narastá s veľkosťou siete, jeho výpočet je komplikovanejší. Posledný činiteľ vzťahu predstavuje vplyv zmeny váh vstupujúcich do skrytého neurónu h_r na jeho výstup. Je možné ho vypočítať pomocou derivácie aktivačnej funkcie ako

$$\frac{\partial h_r}{\partial w_{(h_{r-1}, h_r)}} = h_{r-1} \cdot f'(a_{h_r}) \quad (3.7)$$

Pri výpočte už spomenutého agregovaného člena $\Delta(h_r, o)$ sa využíva fakt, že výpočtový graf siete je acyklický, preto je možné začať výpočet spätných gradientov od výstupnej vrstvy. Najskôr sa vypočíta $\Delta(h_k, o)$ pre poslednú skrytú vrstvu h_k , a následne sa gradienty šíria späť smerom k vstupným vrstvám pomocou rekurzívne definovaného vzťahu. Navyše, každý výstupný neurón je pri výpočte inicializovaný na hodnotu $\frac{\partial \mathcal{L}}{\partial o} = \Delta(o, o)$. Rekurzívny vzťah pre $\Delta(h_r, o)$ sa následne pomocou reťazového pravidla odvodí ako

$$\Delta(h_r, o) = \frac{\partial \mathcal{L}}{\partial h_r} = \sum_{h: h_r \rightarrow h} \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial h_r} = \sum_{h: h_r \rightarrow h} \frac{\partial h}{\partial h_r} \Delta(h, o), \quad (3.8)$$

kde $\Delta(h, o)$ je gradient predchádzajúcej vrstvy (bližšie k výstupu), teda je potrebné odvodiť parciálnu deriváciu $\frac{\partial h}{\partial h_r}$. Na to sa využije skutočnosť, že skryté neuróny h_r a h sú spojené váhou $w_{(h_r, h)}$ a a_h je argument aktivačnej funkcie v neuróne h . Teda pomocou reťazového pravidla jednej premennej je možné odvodiť:

$$\frac{\partial h}{\partial h_r} = \frac{\partial h}{\partial a_h} \cdot \frac{\partial a_h}{\partial h_r} = \frac{\partial f(a_h)}{\partial a_h} \cdot w_{(h_r, h)} = f'(a_h) \cdot w_{(h_r, h)} \quad (3.9)$$

Dosadením vzťahu 3.9 do 3.8 získavame konečný rekurzívny vzťah pre výpočet agregovaného gradientu chyby vzhľadom na aktiváciu neurónu h_r , ktorý umožňuje efektívny výpočet gradientov bez opakovaného prechodu rovnakých ciest v sieti. [1]:

$$\Delta(h_r, o) = \sum_{h: h_r \rightarrow h} f'(a_h) \cdot w_{(h_r, h)} \cdot \Delta(h, o). \quad (3.10)$$

Rekurzívny vzťah dynamického programovania zo vzťahu 3.10 je v skutočnosti možné vypočítať viacerými spôsobmi, v závislosti od toho, ktoré premenné sa použijú ako medzičlánky v reťazovom pravidle. Všetky tieto varianty výpočtu sú ekvivalentné z hľadiska výsledného gradientu získaného spätným šírením chyby. Vo vzťahu 3.6 sa ako premenné medzičlánku reťazového pravidla používajú výstupy skrytých vrstiev h . V literatúre sa však často uvádza alternatívna verzia, kedy sa namiesto nich použijú argumenty aktivačných funkcií neurónov a_h , teda hodnoty vypočítané po lineárnej transformácii, ale ešte pred aplikáciou aktivačnej funkcie. Preto je namiesto rovnice 3.6 možné použiť aj nasledovný variant reťazového pravidla [1]:

$$\frac{\partial \mathcal{L}}{\partial w_{(h_{r-1}, h_r)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial o} \cdot f'(a_o) \cdot \left[\sum_{[h_r, h_{r+1}, \dots, h_k, o] \in \mathcal{P}} \frac{\partial a_o}{\partial a_{h_k}} \prod_{i=r}^{k-1} \frac{\partial a_{h_{i+1}}}{\partial a_{h_i}} \right]}_{\delta(h_r, o) = \frac{\partial \mathcal{L}}{\partial a_{h_r}}} \cdot \underbrace{\frac{\partial a_{h_r}}{\partial w_{(h_{r-1}, h_r)}}}_{h_{r-1}}. \quad (3.11)$$

v ňom je namiesto predošlej „medzipremennej“ $\Delta(h_r, o) = \frac{\partial \mathcal{L}}{\partial h_r}$ použitá $\delta(h_r, o) = \frac{\partial \mathcal{L}}{\partial a_{h_r}}$. Inicializácia počiatočného stupňa rekurzcie je daná ako

$$\delta(o, o) = \frac{\partial \mathcal{L}}{\partial a_o} = f'(a_o) \cdot \frac{\partial \mathcal{L}}{\partial o}. \quad (3.12)$$

Následne je možné opäť použiť reťazové pravidlo viacerých premenných na zostavenie vzťahu pre rekurziu agregovaného gradientu chyby [1]:

$$\delta(h_r, o) = \frac{\partial \mathcal{L}}{\partial a_{h_r}} = \sum_{h: h_r \rightarrow h} \overbrace{\frac{\partial \mathcal{L}}{\partial a_h}}^{\delta(h, o)} \underbrace{\frac{\partial a_h}{\partial a_{h_r}}}_{f'(a_{h_r})w_{(h_r, h)}} = f'(a_{h_r}) \sum_{h: h_r \rightarrow h} w_{(h_r, h)} \cdot \delta(h, o) \quad (3.13)$$

Výsledný gradient účelovej funkcie odchýlky siete je potom vyjadrený nasledujúcim vzťahom:

$$\frac{\partial \mathcal{L}}{\partial w_{(h_{r-1}, h_r)}} = \delta(h_r, o) \cdot h_{r-1} \quad (3.14)$$

Proces tréovania UNS môže prebiehať v dvoch režimoch. Pri *vzorkovom režime* dochádza k úprave váh po privedení každej novej vstupnej vzorky. V *dávkovom režime* sa váhy korigujú až po uskutočnení celej epochy, teda po kompletnom prechode celého trénovacieho súboru cez sieť, s využitím globálnej chyby zo všetkých vzoriek a výstupov.

3.3 Problémy tréovania UNS

Neurónové siete poskytujú silný nástroj pri riešení komplexných a analyticky ťažko odvoditeľných úloh. Aby ich tréovanie nebolo kontraproduktívne, je treba si uvedomiť problémy, ktoré s ním môžu byť spojené. Medzi najzávažnejšie problémy patria *pretrénovanie* a *problém miznúceho gradientu*. Ťažkosti môžu nastať aj pri nedostatočnej konvergencii účelovej funkcie, uviaznutí v lokálnom alebo falošnom optime či nedostatočnom výpočtovom výkone.

3.3.1 Pretrénovanie

Podstata tréovania neurónových sietí, ktoré je príkladom *učenia s učiteľom*, spočíva v tom, že sieť je schopná sa na základe testovacích dát naučiť všeobecné závislosti medzi vstupnými a výstupnými hodnotami, ktoré má byť následne schopná rozlíšiť aj na neznámych dátach. Po ukončení tréovania siete teda vždy prebieha fáza testovania, kedy je na vstup siete privedená množina vzoriek, ktoré sú odlišné od tréovacích.

Pretrénovanie (angl. *overfitting*) je jav, ktorý môže nastať dôsledkom neskorého ukončenia procesu tréovania, nedostatočnej tréovacej množiny dát, či nevhodnej architektúry siete. Prejavuje sa tak, že neurónová sieť síce dosahuje vysokú úspešnosť pri tréovacích dátach, no jej úspešnosť výrazne klesá pri zovšeobecnení na testovacích dátach, ktoré sú odlišné. Tento problém nastáva často napríklad vtedy, keď UNS obsahuje viac neurónov a prepojení, než zodpovedá skutočnej zložitosti riešenej úlohy, či v prípade nedostatočne veľkej testovacej množiny, čo do počtu a rozsahu prvkov vstupných vzoriek. V prvom prípade je dôsledkom, že sieť sa naučí zaznamenať aj také súvislosti vo vstupných dátach, ktoré nie sú všeobecné a súvisia len s danou množinou vstupných dát. V druhom prípade je zase problém v tom, že sieť sa nestihne naučiť všetky vstupno-výstupné závislosti.

Existuje viacero prístupov, ktoré slúžia na minimalizáciu rizika vzniku pretrénovania:

1. *regularizácia* – obmedzuje zložitosť modelu (siete). Do účelovej funkcie je pridaný penalizačný člen, ktorý uprednostňuje menšie hodnoty váh. Dochádza tak k „preriedeniu siete“, keďže obsahuje menšie množstvo významnejších prepojení (s hodnotou ďalej od nuly). Je možné ju interpretovať ako formu biologicky motivovaného „zabúdania“ nepodstatných vzorov.

2. *prispôsobenie architektúry siete* – využíva známe súvislosti vo vstupných dátach. Keď pri návrhu siete berieme do úvahy osobitosti úlohy, ktorú chceme riešiť, je možné zapracovať do architektúry siete také súvislosti, o ktorých vieme, že sa vyskytujú vo vstupných dátach (napr. súvislosť susedných pixelov pri spracovaní obrazu alebo súvislosť nasledujúcich slov pri spracovaní textu). To umožňuje použiť tie isté sady parametrov na spracovanie viacerých častí vstupu, čím sa znižuje počet neznámych parametrov a zjednodušuje model. Príkladom takýchto sietí sú *rekurentné* alebo *konvolučné* n. s.
3. *krížová validácia* (angl. *cross validation*) – zabráňuje sieti naučiť sa nevšeobecné znaky. Zo súboru testovacích vstupov je vyčlenený validačný súbor. Učenie prebieha na zvyšných tréningových dátach, ale účelová funkcia sa paralelne vyhodnocuje aj na validačných dátach. V momente, keď hodnota validačnej účelovej funkcie začne stagnovať alebo dokonca stúpať – čo indikuje, že sieť sa prestala učiť všeobecné znaky a začína sa učiť znaky špecifické pre tréningové dáta – učenie je zastavené (často už po niekoľkých epochách).
4. *výmena šírky za hĺbku* – namiesto veľkého počtu neurónov v jednej vrstve a malému počtu skrytých vrstiev sa preferuje málo neurónov a veľa skrytých vrstiev. Takáto *hlboká sieť* je schopná dosiahnuť rovnakú, a často aj lepšiu úspešnosť, a navyše môže obsahovať menšie množstvo parametrov.

3.3.2 Miznúce a explodujúce gradienty

Problém miznúcich a explodujúcich gradientov býva zvyčajne spojený s tréňovaním hlbokých neurónových sietí a súvisí s využívaním reťazového pravidla (vzťah 3.6) pri spätnom šírení chyby. Keďže pri výpočte celkového gradientu účelovej funkcie dochádza k početnému násobeniu lokálnych gradientov pozdĺž prepojení, zvlášť pri veľkom počte skrytých vrstiev môže spôsobiť buď exponenciálny útlm k nule alebo naopak rýchly nárast gradientu. Oba javy tak spôsobujú nestabilitu procesu tréňovania.

Častou príčinou vzniku miznúceho gradientu býva napríklad použitie aktivačnej funkcie sigmoid. Tá má v celom definičnom obore deriváciu menšiu ako 0,25. Riešením je jej náhrada funkciou ReLU. Okrem tohto riešenia existujú aj pokročilejšie prístupy, ako napríklad [1]:

1. *Adaptívne rýchlosti učenia* – namiesto jednej globálnej rýchlosti učenia α pre celú sieť sa dynamicky prispôsobuje rýchlosť učenia každého parametra na základe historických gradientov. To zvyšuje efektivitu učenia a zlepšuje konvergenciu. Napr. *AdaGrad*, *RMSProp*, *Adam* (v súčasnosti najpoužívanejší). Ich výhodou je dobrá stabilita a rýchle učenie aj v ťažko optimalizovateľných sieťach.

2. *Metóda konjugovaných gradientov* – je vhodná pre nelineárne a veľké optimalizačné úlohy. Namiesto toho, aby sa váhy aktualizovali iba v smere gradientu, táto metóda kombinuje predchádzajúce smery gradientov, aby sa vytvoril nový, efektívnejší smer. Výhodou je, že môže výrazne zrýchliť konvergenciu a vyhnúť sa osciláciám. Na druhej strane, pre výpočtovú náročnosť je vhodnejšia skôr pre menšie siete alebo ako súčasť učenia po dávkach.
3. *Normalizácia po dávkach* (angl. *batch normalization*) – normalizuje vstupy každej vrstvy v rámci jednej dávky tak, aby mali nulový priemer a jednotkovú varianciu. Tým sa potláča nežiaduce kolísanie rozdelenia vstupných hodnôt spôsobené priebežným menením váh vo vrstvách. Výsledkom je rýchlejšia a stabilnejšia konvergencia počas učenia, ako aj zmiernenie problémov s miznúcim gradientom. Normalizácia v rámci dávky tiež často umožňuje použiť vyššiu rýchlosť učenia a znižuje závislosť na počiatočnom nastavení váh. Je však menej účinná pri veľmi malých dávkach alebo v rekurentných sieťach.

3.3.3 Ťažkosti s konvergenciou

Zvlášť v prípade hlbokých neurónových sietí sa vyskytuje problém s konvergenciou účelovej funkcie, keďže veľký počet skrytých vrstiev spôsobuje ťažší tok gradientov cez sieť. Čiastočne tento problém súvisí s miznutím gradientov, ale má svoje osobitné špecifiká [1].

Jedným zo spôsobov riešenia sú tzv. *bránové siete*. Využívajú špecifické neuróny (bunky), obsahujúce adaptívne riadiace mechanizmy (brány), ktoré selektívne kontrolujú tok informácie v sieti. Ich cieľom je zabezpečiť, aby sa relevantné informácie udržali a prenášali počas spracovania vstupu, zatiaľ čo menej dôležité sa potláčajú. Tento prístup je kľúčový najmä pri modelovaní sekvenčných dát (napr. spracovanie textu, videa), kde dlhodobé závislosti často bránia efektívnemu učeniu. Takéto siete sa radia medzi rekurentné neurónové siete a patria tu napr. LSTM či GRU (bližšie o nich v časti 4.2).

Ďalšou možnosťou sú *reziduálne siete*, ktoré zavádzajú reziduálne prepojenia, teda obchádzajúce vetvy, ktoré umožňujú prenášať informáciu aj gradient priamo naprieč viacerými vrstvami bez ich transformácie. Tým sa výrazne znižuje riziko miznutia gradientu a zlepšuje sa stabilita učenia, čo umožňuje trénovať siete s desiatkami až stovkami vrstiev. Tento prístup zároveň urýchľuje konvergenciu a vedie k lepšiemu využitiu kapacity modelu. Známy príklad takejto architektúry je ResNet [6]. Princíp fungovania reziduálnych sietí spočíva v tom, že namiesto učenia sa váh, teda priamej transformácie vstupu na výstup, sa učí len rozdiel medzi vstupom a výstupom. Takéto

prepojenie umožňuje gradientom efektívne pretekať aj do veľmi hlbokých vrstiev siete počas spätného šírenia chyby.

3.3.4 Lokálne a falošné optimá

Optimalizačná funkcia neurónových sietí je silne nelineárna a obsahuje veľké množstvo lokálnych extrémov. Preto je výber vhodnej inicializácie parametrov kľúčový. Nesprávne inicializačné body môžu viesť k nevhodným (lokálnym alebo falošným) minimám. Jednou z metód, ako sa tomu vyhnúť, je tzv. *predtrénovanie* (angl. *pretraining*), ktoré spočíva v postupnom tréovaní jednotlivých vrstiev siete, často pomocou učenia bez učiteľa. Tento prístup poskytuje rozumné počiatočné hodnoty váh, ktoré obchádzajú irelevantné časti parametrového priestoru. Predtrénovanie tiež pomáha znižovať riziko pretrénovania, keďže sa vyhýba minimám, ktoré sú síce vhodné pre tréovaciu množinu, no zlyhávajú na testovacích dátach – tzv. falošným optimám [1].

3.3.5 Výpočtová náročnosť

Jednou z hlavných výziev pri návrhu neurónových sietí je dlhá doba tréovania, ktorá môže v oblastiach ako spracovanie textu či obrazu trvať aj celé týždne. Kľúčový pokrok v tomto smere priniesol vývoj výkonnejšej hardvérovej infraštruktúry, najmä grafických procesorov (GPU), ktoré sú schopné paralelne vykonávať výpočty typické pre neurónové siete. Platformy ako Torch alebo TensorFlow majú vstavanú podporu pre GPU, čo umožňuje efektívnejšie učenie aj zložitých architektúr. Hoci k pokroku v hlbokom učení prispeli aj algoritmické vylepšenia, značná časť úspechu spočíva v tom, že existujúce algoritmy dnes dokážeme lepšie využiť vďaka výkonnejšiemu hardvéru. Výhodou väčšiny neurónových sietí je, že náročné výpočty prebiehajú počas tréovania, zatiaľ čo predikčná fáza je už relatívne rýchla, čo je dôležité pri aplikáciách v reálnom čase [1].

3.4 Súčasné prístupy k tréovaniu

Snaha o vyriešenie vyššie spomenutých problémov tréovania UNS podnietila vznik množstva variácií učiacich algoritmov, ktoré sú síce v základoch postavené na metódach gradientného zostupu a spätného šírenia chyby, ale obsahujú aj ďalšie modifikácie a vylepšenia, týkajúce sa či už len samotného procesu tréovania, alebo aj architektúry celej siete, resp. jej buniek.

Jedným z používaných vylepšení učenia je tzv. *prenos znalostí* (angl. *transfer learning*). Spočíva v tom, že sieť sa netrénuje od nuly, ale vychádza sa z už predtrénovaného modelu, ktorý bol učný na rozsiahlej množine údajov. Tento model sa následne „dolaďuje“ (tzv. *fine-tuning*) na konkrétnu úlohu pomocou menšieho množstva nových

dát. Transferové učenie výrazne znižuje nároky na dáta aj výpočtový čas a zároveň zvyšuje kvalitu výsledkov, najmä ak sa pracuje s málo rozsiahlymi alebo špecializovanými údajmi pre konkrétnu oblasť. Táto metóda je dnes základom mnohých aplikácií – od spracovania prirodzeného jazyka (napr. BERT, GPT) až po analýzu obrazov (napr. ResNet, EfficientNet).

Ďalším riešením pre zníženie výpočtovej náročnosti je *trénovanie so zmiešanou číselnou presnosťou* (angl. *mixed precision*). V tomto prístupe prebiehajú niektoré výpočty počas tréovania s nižšou číselnou presnosťou (napr. 16-bitové čísla namiesto štandardných 32-bitových). Výsledkom je nižšia pamäťová záťaž a výrazne rýchlejšie výpočty, najmä na grafických procesoroch (GPU) a špecializovaných akcelerátoroch (TPU), ktoré sú pre takéto výpočty optimalizované. Táto technika si zachováva presnosť tréovania pomocou špeciálnych mechanizmov (napr. škálovanie gradientov) a dnes je často využívaná v priemyselnej praxi.

Trénovanie pomocou klasickej metódy gradientného zostupu býva pri zložitejších architektúrach neefektívne, pretože používa konštantnú rýchlosť učenia pre všetky parametre siete po celú dobu procesu tréovania. Veľká rýchlosť je totiž dobrá na rýchle priblíženie sa do blízkosti dobrého riešenia, avšak zväčša neumožňuje jemné doladenia sa ku optimu, teda sa môže stať, že algoritmus tréovania ostane oscilovať v okolí optimálneho bodu alebo dokonca zdiverguje do nestabilnej oblasti. Na druhej strane, voľba malej rýchlosti učenia má za následok veľmi pomalú konvergenciu učenia. To sú dôvody pre vznik tzv. adaptívnych optimalizačných algoritmov, ktoré rýchlosť učenia dynamicky prispôbujú pre každý parameter siete zvlášť počas celého procesu tréovania.

Jedným z najrozšírenejších algoritmov pre tréovanie hlbokých neurónových sietí v súčasnosti je Adam (Adaptive Moment Estimation) [12]. Využíva adaptívne nastavovanie parametrov siete (váh a biasov) na základe priebežných štatistík gradientu, ich prvého momentu (priemeru), ktorý informuje o „smerovaní“ optimalizácie a druhého momentu (rozptylu). Zároveň využíva tzv. L2 regularizačný člen, ktorý má za cieľ udržiavať hodnoty váh blízke nule (angl. *weight decay*). Výsledný vzorec pre aktualizáciu i -teho parametru má tvar

$$w_i \leftarrow w_i - \frac{\alpha_t}{\sqrt{A_i}} F_i; \quad \forall i, \quad (3.15)$$

kde agregovaný člen prvého momentu F_i , agregovaný člen druhého momentu A_i a rýchlosť učenia α_t sú priebežne aktualizované na základe nasledujúcich vzťahov [1]:

$$F_i \leftarrow \rho_f F_i + (1 - \rho_f) \left(\frac{\partial \mathcal{L}}{\partial \Xi_i} \right); \forall i, \quad (3.16a)$$

$$A_i \leftarrow \rho A_i + (1 - \rho) \left(\frac{\partial \mathcal{L}}{\partial \Xi_i} \right)^2; \forall i, \quad (3.16b)$$

$$\alpha_t = \alpha \left(\frac{\sqrt{1 - \rho^t}}{1 - \rho_f^t} \right), \quad (3.16c)$$

pričom $\rho, \rho_f \in (0, 1)$ sú voliteľné parametre. Novšia verzia algoritmu, AdamW [16] vylepšuje pôvodný Adam v tom, že regularizačný člen nemieša s výpočtom gradientného momentu, ale počíta ho osobitne, čo vedie k stabilnejšiemu trénovaniu a lepšiemu generalizačnému výkonu siete.

4 Hlboké učenie

Hlboké učenie (angl. *deep learning*) predstavuje jednu z najdôležitejších súčasných oblastí vývoja UI. Prekonáva ťažkosti klasických perceptrónových sietí v riešení komplexnejších úloh využitím viacvrstvových neurónových architektúr, kde každá vrstva extrahuje čoraz abstraktnejšie znaky z výstupov predchádzajúcich vrstiev. Vďaka tejto hierarchickej reprezentácii je možné modelovať zložité funkcie priamo z dát bez nutnosti manuálne definovaných príznakov [15]. Hlboké siete tak výrazne prevyšujú tradičné MLP v schopnosti učiť sa reprezentácie priamo zo surových dát, čím umožňujú riešiť zložité úlohy ako rozpoznávanie objektov, preklad jazykov alebo predikciu dynamiky systémov.

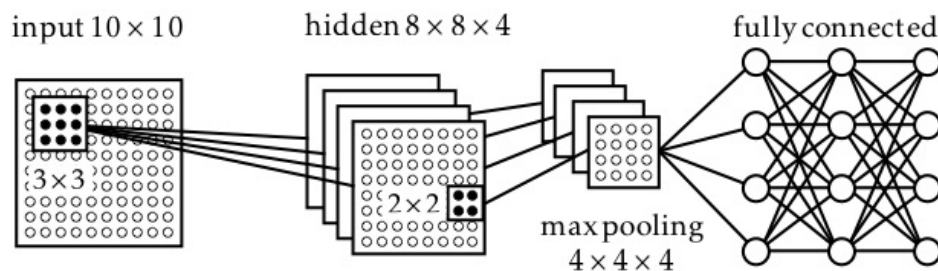
Na rozdiel od plytkých sietí vyžaduje trénovanie hlbokých modelov pokročilejšie optimalizačné techniky. Kým MLP siete často využívali obyčajný gradientný zostup s fixnou rýchlosťou učenia, hlboké siete zvyknú používať algoritmy ako Adam, RMSProp alebo momentové metódy, ktoré zohľadňujú historické hodnoty gradientov a adaptívne upravujú parametre učenia [12]. Za účelom predchádzania vyššie spomenutým problémom, ako je miznúci a vybuchujúci gradient, sa zároveň používajú metódy ako regularizácia (napr. L2 penalizácia alebo „dropout“), váhová inicializácia a plánovanie rýchlosti učenia [5].

4.1 Konvolučné neurónové siete

Konvolučné neurónové siete (CNN) sú špeciálne navrhnuté na efektívne spracovanie dát so štruktúrou v priestore. Hoci boli vynájdené ešte koncom 20. storočia, do širšieho používania sa dostali až po práci kolektívu A. Krizhevsky [13] a dodnes sú najúspešnejším nástrojom na rozpoznávanie a klasifikáciu obrazu z kamery. Oproti klasickým plne prepojeným sieťam majú pri rovnakom počte neurónov menej prepojení, vďaka čomu sú jednoduchšie na natrénovanie a odolnejšie voči pretrénovaniu. Princíp ich fungovania je inšpirovaný fungovaním mozgu cicavcov pri spracovaní zrakových vnemov [18].

Vstupom do CNN je obvykle obraz vo formáte dvojrozmernej bitmapy. Pri rozpoznávaní obrazu sa v niekoľkých iteráciách opakuje postup, kedy sa na vstupnú vrstvu neurónov aplikujú konvolučné filtre, tzv. *jadrá* (angl. *kernels*), pričom každé z nich obsahuje počet neurónov rádovo nižší než je rozmer vstupnej vrstvy (napr. 3×3 , 5×5 , 8×8 , ...). Jadro postupne aplikované na celú vstupnú vrstvu, pričom sa vykonáva operácia konvolúcie. Výstup jedného vstupného neurónu tak možno vyjadriť ako

$$V(i, j) = f \left(\sum_{p=1}^P \sum_{q=1}^Q x(i+p-1, j+q-1) \cdot w(p, q) - \theta \right), \quad (4.1)$$



Obr. 8: Schéma architektúry konvolučnej neurónovej siete. [18]

kde i, j sú súradnice vstupného neurónu, $w(p, q)$ je hodnota jedného neurónu jadra s rozmermi $P \times Q$ a $x(i + p, j + q)$ je vstupná hodnota v okolí daného vstupného neurónu a $f(\cdot)$ predstavuje nelineárnu aktivačnú funkciu (obvykle ReLU) [18].

V procese učenia sú váhy jadier nastavené tak, aby rozpoznávali určité príznaky vo vstupných dátach (napr. hrany, textúry, objekty), čím vznikajú tzv. *mapy príznakov* (angl. *feature maps*). Vlastnosťou tejto techniky je invariantnosť voči posunu, zmene mierky alebo iným formám skreslenia, vďaka čomu je použitím jedného malého jadra možné detegovať ten istý príznak vo viacerých častiach vstupu bez zmeny jeho hodnôt. Významne sa tak redukuje počet parametrov potrebných na natrénovanie takejto siete oproti plnému prepojeniu. Tento proces sa opakuje vo viacerých po sebe nasledujúcich *konvolučných vrstvách*, pričom platí, že čím viac vrstiev sa použije, tým komplexnejšie príznaky je možné rozpoznávať. Na učenie konvolučných vrstiev sa používa algoritmus spätného šírenia chyby [18].

Medzi jednotlivé konvolučné vrstvy sa umiestňujú tzv. *zhlukovacie vrstvy* (angl. *pooling layers*). Ich vstupom sú výstupy z viacerých susedných máp príznakov a redukuje ich na jediné číslo. Tieto vrstvy neobsahujú žiadne parametre, ktoré by boli predmetom tréningu. Úlohou je podvzorkovanie mapy príznakov, na čo sa obvykle využíva funkcia maxima alebo priemeru. Výstup zo zhlukovacích vrstiev slúži ako vstup pre nasledujúce konvolučné vrstvy. Napokon, výsledná klasifikácia CNN môže byť vykonaná pomocou *plne prepojených vrstiev*.

Medzi najvyužívanejšie implementácie patria TensorFlow a PyTorch, ktoré ponúkajú predpripravené moduly na tvorbu a tréning CNN, ako aj množstvo predtrénovaných modelov (napr. VGG, ResNet, MobileNet či EfficientNet) vhodných na prenos znalostí. Tieto knižnice podporujú aj hardvérovú akceleráciu (napr. pomocou GPU), čo umožňuje efektívne tréning rozsiahlych sietí. V praxi sa CNN implementácie uplatňujú v prostrediach ako OpenCV, ktoré ponúka jednoduché rozhranie na inferenciu CNN modelov v reálnom čase a je často využívaný vo vnorených systémoch a robotike. Vďaka týmto nástrojom je dnes práca s CNN prístupná aj bez potreby vlastnej implementácie architektúr od základu.

4.2 Rekurentné neurónové siete

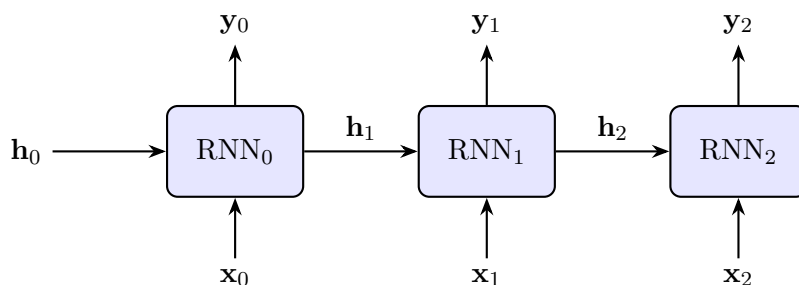
Rekurentné neurónové siete (RNN) tvoria triedu architektúr umelých neurónových sietí navrhnutých špecificky na spracovanie sekvenčných dát. Na rozdiel od bežných dopredných sietí, ktoré spracúvajú vstupy ako nezávislé a nemajú vnútornú pamäť, RNN obsahujú mechanizmus *rekurencie*, ktorý im umožňuje uchovávať informácie o predchádzajúcich vstupoch v tzv. *skrytom stave*. Táto schopnosť ich predurčuje na úlohy, v ktorých poradie alebo časový kontext zohráva rozhodujúcu úlohu – napríklad pri spracovaní textu, zvuku, signálov zo senzorov či trajektórií v priestore.

Tento typ neurónových sietí neobsahuje iba dopredné väzby, teda zľava doprava, ale obsahujú tiež spätné väzby a pamäťové informácie o stave buniek. Zásadným rozdielom RNN oproti klasickým dopredným sieťam je tiež to, že vstupy nie sú pripojené priamo na prvú skrytú vrstvu, ale interagujú so skrytými stavmi, ktoré boli vypočítané na základe predošlých vstupov. Skrytý stav siete sa mení v čase a je reprezentovaný vektorom h_t , ktorého výpočet je funkciou aktuálneho vstupného vektora X_t a predošlého skrytého stavu h_{t-1} :

$$H_t = \sigma(W_{xh} X_t + W_{hh} H_{t-1} + b_h), \quad (4.2)$$

kde W_{xh} a W_{hh} sú váhové matice pre vstup a skrytý stav, b_h je bias a σ je aktivačná funkcia (najčastejšie hyperbolický tangens alebo ReLU). Na výpočet výstupu siete sa potom väčšinu používa aktivačná funkcia softmax alebo sigmoid. Výstup siete môže byť generovaný za každou vstupnou vzorkou alebo až po ukončení jednej dávky. Aktivačné funkcie sú rovnaké pre každú vstupnú vzorku.

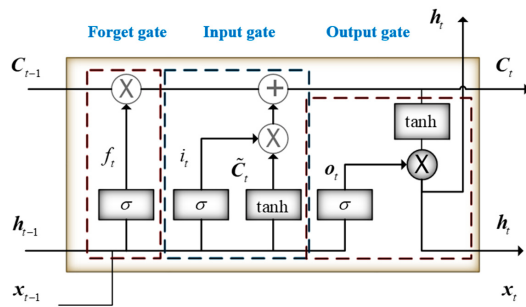
Vďaka schopnosti modelovať dlhodobé závislosti v čase sú RNN vhodné na spracovanie senzorických dát, kde každé meranie tvorí súčasť väčšej sekvencie. Môžu slúžiť aj ako časový filter, nakoľko dokážu integrovať informácie z predchádzajúcich meraní a korigovať neistotu v aktuálnych pozorovaniach. K ich výhodám patrí aj prispôsobivosť a univerzálnosť, keďže tá istá architektúra sa dá použiť na rôzne typy sekvencií. Sú



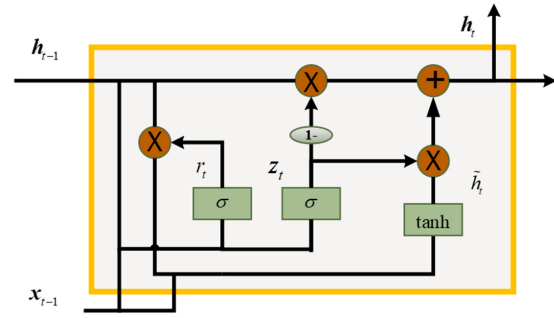
Obr. 9: Rekurentná neurónová sieť – znázornenie toku informácií v čase.

však náročné na tréningovanie a citlivé na dĺžku a kvalitu sekvencie. Medzi najčastejšie používané implementácie RNN patria tieto architektúry:

- *Dlhá krátkodobá pamäť (LSTM)* [7] – (obr. 10a) zavádza vnútorný pamäťový stav bunky a sústavu brán, ktoré regulujú tok informácií: vstupná brána rozhoduje, čo nové si zapamätať, výstupná brána rozhoduje, ktorú informáciu z pamäte použiť ako výstup a zabúdacia brána rozhoduje, čo z pamäte vyhodíť. Prináša teda schopnosť udržať si informáciu z minulosti bez toho, aby došlo k miznutiu gradientu.
- *Rekurentná jednotka s bránou (GRU)* [9] – (obr. 10b) ide o zjednodušenú verziu LSTM, ktorá obsahuje len dve brány: aktualizácia – riadi, koľko z predchádzajúceho stavu sa preniesie ďalej – a resetovacia – rozhoduje, koľko informácie z minulosti ignorovať. Je výpočtovo efektívnejšia než LSTM, no výkonovo porovnateľná.



(a) LSTM sieť – zabúdacia, vstupná a výstupná brána



(b) GRU sieť – aktualizácia a resetovacia brána

Obr. 10: Porovnanie štruktúr LSTM a GRU rekurentných neurónových sietí. [10]

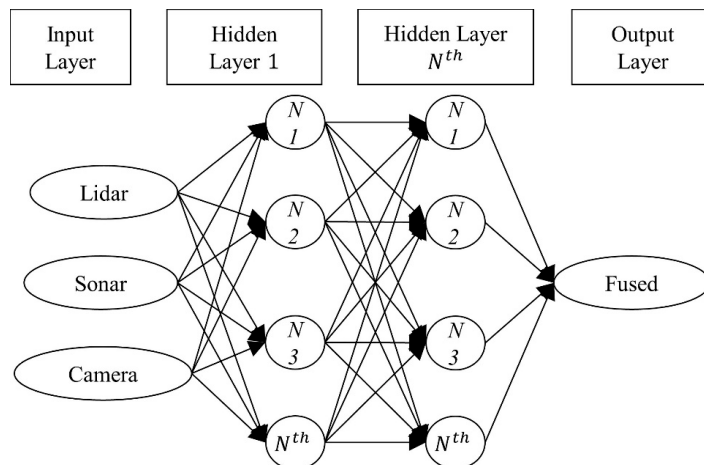
5 Fúzia lokalizácie pomocou UNS

Presná znalosť polohy v prostredí – lokalizácia – predstavuje jeden zo základných predpokladov pre spoľahlivú prevádzku a efektívne plnenie úloh mobilného robota. Na tento účel sa bežne využíva viacero typov senzorov, ako napríklad GNSS, INS, vizuálne systémy alebo laserové snímače. Každý z týchto senzorov však podlieha určitým obmedzeniam a v praxi neposkytuje dokonale presné ani spoľahlivé údaje. Na zníženie neistoty a zvýšenie robustnosti odhadu aktuálnej polohy sa preto využíva fúzia údajov, t. j. inteligentná integrácia viacerých dátových tokov do jedného spoločného odhadu, ktorý je spravidla presnejší a konzistentnejší než jednotlivé zdroje samostatne.

Tradičné prístupy k fúzii údajov sú založené prevažne na pravdepodobnostných modeloch (napr. Bayesovská filtrácia, Kalmanove filtre, časticové filtre) alebo metódach intervalového odhadu. Najväčšou výzvou pri použití týchto metód je však získanie presného modelu systému. V prípade komplexných a vysoko dynamických systémov môže byť odvodenie presného modelu zvlášť náročné. Navyše, senzorické dáta bývajú sprevádzané vlastnou nepresnosťou a neistotou, ktorých zapracovanie do modelu je problematické. Vo výsledku tak nepresný model systému môže vážne obmedziť úspešnosť analytických metód fúzie [4].

S nástupom moderných techník umelej inteligencie – predovšetkým umelých neurónových sietí a hlbokého učenia – sa postupne presadzuje aj ich uplatnenie v tejto oblasti. Tieto modely sú schopné spracovávať veľké objemy vysoko dimenzionálnych dát, učiť sa zložené nelineárne vzťahy medzi jednotlivými vstupmi a zohľadňovať aj ich časové závislosti bez hlbokej analytickej znalosti modelu, čo z nich robí atraktívny nástroj pre riešenie úloh lokalizácie prostredníctvom senzorickej fúzie.

Príkladom využitia štandardnej, plne prepojenej DFFN siete na fúziu lokalizácie môže byť práca Barreto-Cuberto a kol [2]. Využíva sa tu kombinácia troch typov senzorov: 360-stupňový 2D lidar, stereokamera a ultrazvukové senzory. Úlohou neurónovej siete bolo zlepšiť schopnosť mobilného robota detegovať rôzne druhy prekážok vrátane sklenených povrchov, ktoré sú pre lidar často neviditeľné. Každý z týchto senzorov má iné výhody: ultrazvuk vie spoľahlivo detegovať sklo, stereo kamera zachytáva trojrozmernú hĺbkovú štruktúru a lidar poskytuje vysoké rozlíšenie v 2D rovine. Pred fúziou sa surové senzorické dáta upravujú – aplikujú sa filtre (napr. Kalmanov filter na ultrazvuk), transformácie do spoločného súradnicového systému a projekcia mračien bodov na 2D rovinu. Neurónová sieť bola trénovaná na predikciu spoľahlivých vzdialeností k prekážkam spojením vstupov zo všetkých troch senzorov. Používa Adam optimalizátor, aktivačnú funkciu ReLU a ďalšie osvedčené techniky pre nelineárne dáta. Výsledkom fúzie je jednotná metrika vzdialeností, ktorú systém využíva na tvorbu 2D okupačnej mapy



Obr. 11: Fúzia senzorov pomocou DFFN [2]

(angl. Occupancy Grid Map). Experimentálne porovnanie viacerých stratégií (lidar-sólo, lidar+sonar, lidar+kamera+sonar) ukázalo, že fúzia pomocou UNS výrazne znižuje strednú kvadratickú chybu (RMSE) (až pod 3 cm), pričom samotný lidar mal chybu cez 2 metre pri prítomnosti skla. Navyše, kombinácia kamery a sonaru umožňuje rozpoznať aj objekty pod úrovňou skenovania lidar, čím sa zvyšuje bezpečnosť a spoľahlivosť navigácie.

5.1 CNN fúzia

Nakoľko je architektúra CNN sietí prispôbená na spracovanie dvojrozmerných dát, v súvislosti s fúziou sa ich využitie objavuje najčastejšie pri spracovaní kamerového obrazu, príp. máp.

Jednou z prvých implementácií metód hlbokého učenia na účely fúzie odometrie vôbec je práca autorov Valente a kol. z roku 2019 [24], ktorí sa zaoberali fúziou monokamery a 2D laserového skenera. Cieľom bolo eliminovať závislosť od presnej kalibrácie medzi senzormi a zložitosť návrhu ručne definovaných fúzných pravidiel. Navrhnutý prístup transformuje klasický regresný problém odometrie (odhadu vzdialenosti a natočenia medzi snímkami) na problém ordinálnej klasifikácie, čím zjednodušuje učenie siete. Vstupom do siete sú vždy dvojice po sebe idúcich 360° laserových skenov a obrázkov z kamery, pričom sieť predikuje transformáciu $(\Delta d, \Delta \theta)^T$ – teda prejdenú vzdialenosť a rotačný uhol medzi dvoma časovými krokmi. Akumuláciou týchto lokálnych odhadov je možné rekurzívne rekonštruovať globálnu trajektóriu robota. Architektúra pozostáva z dvoch vetiev (CNN pre laser, CNN pre kameru), ktorých výstupy sú zlúčené a prevedené na odhad translácie a rotácie. Výsledky na reálnych dátach z KITTI datasetu ukazujú, že konvolučné siete dokážu efektívne fúzovať heterogénne senzorické údaje pre účely

lokalizácie v reálnom čase. Tento prístup je preto vhodný najmä v situáciách, kde nie sú dostupné kinematické dáta (napr. z enkodérov) alebo GNSS.

Efektívne využitie CNN technológie je predstavené v článku [25], kde autori predstavujú systém Dynamic-SLAM, ktorý rozširuje klasické vizuálne SLAM riešenia tak, aby dokázali spoľahlivo pracovať v dynamickom prostredí. Kým tradičné SLAM metódy predpokladajú, že všetky pozorované prvky sú statické, tento prístup využíva sémantickú segmentáciu pomocou SSD konvolučnej neurónovej siete na rozpoznávanie dynamických objektov v obraze. Tieto objekty sa detegujú v samostatnom vlákne a na základe toho sa ich zodpovedajúce obrazové prvky (napr. rohové body) vylúčia z odhadu pohybu robota, čím sa výrazne znižuje chyba lokalizácie. Keďže SSD môže niektoré objekty vynechať, autori navrhujú kompenzačný algoritmus, ktorý využíva fakt, že pohyb objektov medzi po sebe idúcimi snímkami býva plynulý – tzv. rýchlostnú invariantnosť – a na základe toho spätne identifikuje nedetekované objekty. Následne sa v sledovacom vlákne aplikuje selektívne sledovanie, ktoré spracúva len spoľahlivé statické prvky. Výsledný systém je robustnejší voči narušeniu dynamikou prostredia, funguje v reálnom čase, a experimentálne preukazuje vyššiu presnosť a úspešnosť lokalizácie než existujúce riešenia ako ORB-SLAM2 či DynaSLAM, a to nielen v simulovaných testoch, ale aj na mobilnej robotikej platforme v reálnom svete.

DeepFusion je riešenie predstavené v práci [14], ktoré systém pre hustú 3D rekonštrukciu z monokulárneho videa. Kombinuje výhody geometrických metód a hlbokého učenia. Motiváciou je prekonanie obmedzení existujúcich metód, ako je obmedzený dosah hĺbkových kamier a nejednoznačnosť mierky pri rekonštrukcii z fotometrických chýb, čo je kľúčové pre robotické úlohy. Na rozdiel od klasických SLAM systémov, ktoré vytvárajú len riedke mapy na základe kľúčových bodov, DeepFusion generuje husté hĺbkové mapy v mierke pomocou CNN, ktorá predikuje nielen hĺbku z jedného obrazu, ale aj jej gradienty a neistoty. Tieto výstupy sa potom pravdepodobnostne spájajú so čiastočne hustou rekonštrukciou z viacerých pohľadov, čím sa zachováva globálna konzistencia aj absolútna mierka. Optimalizácia prebieha priebežne počas prechodu novými snímkami, ale sieť sa spúšťa len raz pre každý kľúčový snímok, čím sa zachováva reálny čas spracovania na GPU. Tento hybridný prístup využíva prednosti učenia (presnosť vo vnútri objektov) aj geometrie (ostré hrany) a podľa experimentov dosahuje porovnateľné alebo lepšie výsledky ako iné existujúce systémy.

5.2 RNN fúzia

Ako už bolo spomenuté v časti 4.2, silnou stránkou RNN sietí je schopnosť spracúvať časovo-závislé dáta a pamätať si z nich vzniknuté závislosti. Vďaka tomu je táto architektúra vhodná na spracovanie meraní z GNSS a IMU.

Autori článku [3] reagujú na obmedzenia tradičných metód integrovanej navigácie INS/GNSS, najmä rozšíreného Kalmanovho filtra (EKF), ktorý si vyžaduje presný model chýb senzorov a trpí zníženou presnosťou pri dlhodobom výpadku GNSS signálu v silne nelineárnych alebo rušených prostrediach. Na rozdiel od statických neurónových sietí, ktoré nedokážu zachytiť časové závislosti chýb INS, navrhujú autori využitie RNN schopnej modelovať dynamické väzby medzi aktuálnymi a minulými chybami INS. RNN sa trénuje na dátach z obdobia, keď je GNSS dostupné, a následne dokáže počas výpadku GNSS predikovať chyby polohy a rýchlosti INS na základe vlastnej „pamäte“ a údajov o dynamike vozidla (napr. zmeny rýchlosti a orientácie). Tento prístup umožňuje kontinuálne a presné určovanie polohy bez potreby zložitého modelovania alebo ladenia parametrov ako v prípade EKF. Výsledky zo simulácií aj reálnych experimentov (napr. počas 10-hodinovej plavby lode) ukázali, že navrhovaná metóda dosahuje výrazné zníženie chýb v porovnaní s EKF (až o 61 % v polohe a 40 % v rýchlosti) aj s ELM, čo potvrdzuje jej potenciál pre navigáciu v prostrediach s nedostupným alebo rušeným GNSS signálom.

Podobný problém súvisiaci s potenciálnym obmedzením dostupnosti a presnosti GNSS signálu v zastavaných mestských oblastiach či tuneloch sa rozhodli riešiť aj autori práce [11]. Na preklopenie výpadkov GNSS meraní sa rozhodli zapojiť do kombinácie aj údaje z IMU a kolesovej odometrie, no v snahe vyhnúť sa potrebe komplexného odhadu modelu jednotlivých senzorov využili LSTM sieť. Model predikuje aktuálnu polohu na základe predchádzajúcich meraní zo všetkých spomenutých senzorov. Autori uvádzajú 40%-tné zmenšenie chyby odhadu v porovnaní so samostatnou GNSS navigáciou v zjednodušených testovacích podmienkach. Napriek ukázanému potenciálu metód hlbokého učenia v podobe obídienia procesu kalibrácie a modelovania, riešenie by ešte potrebovalo validáciu v náročnejších reálnych podmienkach.

Rekurzívne neurónové siete je možné využiť nie len na nahradenie analytických prístupov fúzie, ale aj na ich doplnenie, resp. riešenie problémov, na ktoré štandardné metódy nie sú stavané. Príkladom je nedávno publikovaná práca [8], v ktorej je fúzia pomocou EKF doplnená o RNN sieť za účelom riešenia asynchrónnych frekvencií jednotlivých senzorov, akumulácie driftu a šumu merania. Algoritmus prebieha v troch krokoch. V prvom je fúzia údajov z IMU a kolesovej odometrie zabezpečená pomocou EKF, následne je použitá RNN sieť na modelovanie časových závislostí, čo umožňuje vyhladzovanie dát a redukciu driftu a šumu, ktoré EKF sám o sebe nedokáže dobre potlačiť, najmä pri nelineárnych dynamikách alebo asynchrónnom snímaní. RNN tak vystupuje ako korekčný mechanizmus, ktorý spracúva už EKF-fúzované dáta a zohľadňuje dlhodobéjšie závislosti v signáli. Následne sa výsledok koriguje pomocou komplementárnej fúzie s lidarom pre zvýšenie robustnosti a spoľahlivosti. Vďaka tejto architektúre

algoritmus dosahuje lokalizačnú chybu do 8 cm a prekračuje výkon metód ako časticový filter alebo SLAM založený na grafovej optimalizácii polôh (tzv. Graph SLAM) pri zachovaní výpočtovej efektivity, čím je vhodný aj pre reálne aplikácie v autonómnej navigácii.

Záver

Technológie umelých neurónových sietí a hlbokého učenia zohrávajú čoraz dôležitejšiu úlohu v oblasti vnímania a lokalizácie autonómnych vozidiel. Oblasť fúzie senzorických dát nepatrí medzi tradičné príklady implementácie metód umelej inteligencie, vývoj v nej sa dlho uberal skôr analytickými a pravdepodobnostnými smermi. Problém s náročnosťou modelovania a analytického či pravdepodobnostného spracovania senzorických dát v súlade s rozvojom umelej inteligencie podnecuje rozvoj aplikácií aj v týchto kombináciách. UNS majú potenciál zohrať úlohu nie len v samotnom spájaní senzorických tokov do jedného, ale najmä v inteligentnom predspracovaní senzorických dát za účelom potlačenia šumov či iných nežiadúcich faktorov. Zlepšenie lokalizácie by bolo možné tiež dosiahnuť detekciou výrazných orientačných bodov (napr. značky, stĺpy v exteriéri alebo dvere, schodiská v interiéri) a ich porovnávaním s mapou, pričom využitie 3D neurónových sietí zvyšuje robustnosť voči rušivým vplyvom prostredia. Kým konvolučné siete sú dnes štandardom pri spracovaní obrazov, v lokalizácii a mapovaní je potenciál iných architektúr, najmä rekurentných, stále nevyužitý. Budúci výskum sa preto môže zamerať na vytváranie komplexnejších riešení a zároveň posilniť spoľahlivosť, opakovateľnosť a odolnosť voči útokom kombinovaním dátovo orientovaných prístupov s klasickými modelmi.

Literatúra

1. AGGARWAL, C. C. *Neural Networks and Deep Learning: A Textbook* [online]. Cham: Springer International Publishing, 2018 [cit. 2025-07-14]. ISBN 978-3-319-94462-3 978-3-319-94463-0. Dostupné z DOI: 10.1007/978-3-319-94463-0.
2. BARRETO-CUBERO, A. J.; GÓMEZ-ESPINOSA, A.; ESCOBEDO CABELLO, J. A.; CUAN-URQUIZO, E.; CRUZ-RAMÍREZ, S. R. Sensor Data Fusion for a Mobile Robot Using Neural Networks. *Sensors* [online]. 2022, **22**(1), 305 [cit. 2025-07-04]. ISSN 1424-8220. Dostupné z DOI: 10.3390/s22010305.
3. DAI, H.-f.; BIAN, H.-w.; WANG, R.-y.; MA, H. An INS/GNSS Integrated Navigation in GNSS Denied Environment Using Recurrent Neural Network. *Defence Technology* [online]. 2020, **16**(2), 334–340 [cit. 2025-07-23]. ISSN 2214-9147. Dostupné z DOI: 10.1016/j.dt.2019.08.011.
4. FAYYAD, J.; JARADAT, M. A.; GRUYER, D.; NAJJARAN, H. Deep Learning Sensor Fusion for Autonomous Vehicle Perception and Localization: A Review. *Sensors* [online]. 2020, **20**(15), 4220 [cit. 2025-07-23]. ISSN 1424-8220. Dostupné z DOI: 10.3390/s20154220.
5. GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge, Mass.: MIT Press, 2016. ISBN 978-0-262-03561-3. Dostupné tiež z: <http://www.deeplearningbook.org>.
6. HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep Residual Learning for Image Recognition. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* [online]. Las Vegas, NV, USA: IEEE, 2016, s. 770–778 [cit. 2025-07-20]. Dostupné z DOI: 10.1109/cvpr.2016.90.
7. HOCHREITER, S.; SCHMIDHUBER, J. Long Short-Term Memory. *Neural Computation* [online]. 1997, **9**(8), 1735–1780 [cit. 2025-07-20]. ISSN 0899-7667. Dostupné z DOI: 10.1162/neco.1997.9.8.1735.
8. HUANG, Z.; YE, G.; YANG, P.; YU, W. Application of Multi-Sensor Fusion Localization Algorithm Based on Recurrent Neural Networks. *Scientific Reports* [online]. 2025, **15**(1), 8195 [cit. 2025-07-23]. ISSN 2045-2322. Dostupné z DOI: 10.1038/s41598-025-90492-4.
9. CHUNG, J.; GULCEHRE, C.; CHO, K.; BENGIO, Y. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling* [online]. 2014-12-11. [cit. 2025-07-20]. Dostupné z DOI: 10.48550/arXiv.1412.3555. Comment: Presented in NIPS 2014 Deep Learning and Representation Learning Workshop.

10. JIANG, C. et al. A Mixed Deep Recurrent Neural Network for MEMS Gyroscope Noise Suppressing. *Electronics* [online]. 2019, **8**(2), 181 [cit. 2025-07-23]. ISSN 2079-9292. Dostupné z DOI: 10.3390/electronics8020181.
11. KIM, H.-U.; BAE, T.-S. Deep Learning-Based GNSS Network-Based Real-Time Kinematic Improvement for Autonomous Ground Vehicle Navigation. *Journal of Sensors* [online]. 2019, **2019**(1), 3737265 [cit. 2025-07-23]. ISSN 1687-7268. Dostupné z DOI: 10.1155/2019/3737265.
12. KINGMA, D. P.; BA, J. *Adam: A Method for Stochastic Optimization* [online]. 2017-01-30. [cit. 2025-07-21]. Dostupné z DOI: 10.48550/arXiv.1412.6980. Comment: Published as a conference paper at the 3rd International Conference for Learning Representations, San Diego, 2015.
13. KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. ImageNet Classification with Deep Convolutional Neural Networks. In: *Advances in Neural Information Processing Systems* [online]. Curran Associates, Inc., 2012, zv. 25 [cit. 2025-07-21]. Dostupné z: https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.
14. LAIDLOW, T.; CZARNOWSKI, J.; LEUTENEGGER, S. *DeepFusion: Real-Time Dense 3D Reconstruction for Monocular SLAM Using Single-View Depth and Gradient Predictions* [online]. 2022-07-25. [cit. 2025-07-23]. Dostupné z DOI: 10.48550/arXiv.2207.12244. Comment: Accepted at ICRA 2019.
15. LECUN, Y.; HINTON, G. E.; BENGIO, Y. Deep Learning. *Nature* [online]. 2015 [cit. 2025-07-14]. Dostupné z DOI: 10.1038/nature14539.
16. LOSHCHILOV, I.; HUTTER, F. *Decoupled Weight Decay Regularization* [online]. 2019-01-04. [cit. 2025-07-21]. Dostupné z DOI: 10.48550/arXiv.1711.05101. Comment: Published as a conference paper at ICLR 2019.
17. MCCULLOCH, W. S.; PITTS, W. A Logical Calculus of the Ideas Immanent in Nervous Activity. *The bulletin of mathematical biophysics* [online]. 1943, **5**(4), 115–133 [cit. 2025-07-05]. ISSN 1522-9602. Dostupné z DOI: 10.1007/BF02478259.
18. MEHLIG, B. *Machine Learning with Neural Networks* [online]. 2021. [cit. 2025-07-05]. Dostupné z DOI: 10.1017/9781108860604. Comment: Revised version, 241 pages.
19. MINSKY, M.; PAPERT, S. A. *Perceptrons: An Introduction to Computational Geometry* [online]. The MIT Press, 2017 [cit. 2025-07-18]. ISBN 978-0-262-34393-0. Dostupné z DOI: 10.7551/mitpress/11301.001.0001.

20. ROSENBLATT, F. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*. 1958, **65**(6), 386–408. ISSN 1939-1471. Dostupné z DOI: 10.1037/h0042519.
21. RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning Representations by Back-Propagating Errors. *Nature* [online]. 1986, **323**(6088), 533–536 [cit. 2025-07-14]. ISSN 1476-4687. Dostupné z DOI: 10.1038/323533a0.
22. RUSSELL, S. J.; NORVIG, P.; DAVIS, E. *Artificial Intelligence: A Modern Approach*. 3rd ed. Upper Saddle River: Prentice Hall, 2010. Prentice Hall Series in Artificial Intelligence. ISBN 978-0-13-604259-4.
23. SCHMIDHUBER, J. Deep Learning in Neural Networks: An Overview. *Neural Networks* [online]. 2015, **61**, 85–117 [cit. 2025-07-14]. ISSN 0893-6080. Dostupné z DOI: 10.1016/j.neunet.2014.09.003.
24. VALENTE, M.; JOLY, C.; FORTELLE, A. d. L. *Deep Sensor Fusion for Real-Time Odometry Estimation* [online]. 2019-07-31. [cit. 2025-07-04]. Dostupné z DOI: 10.48550/arXiv.1908.00524. Comment: arXiv admin note: substantial text overlap with arXiv:1902.08536.
25. XIAO, L.; WANG, J.; QIU, X.; RONG, Z.; ZOU, X. Dynamic-SLAM: Semantic Monocular Visual Localization and Mapping Based on Deep Learning in Dynamic Environment. *Robotics and Autonomous Systems* [online]. 2019, **117**, 1–16 [cit. 2025-07-23]. ISSN 0921-8890. Dostupné z DOI: 10.1016/j.robot.2019.03.012.